

Import statement

```
1 from math import pi
2 tau = 2 * pi
```

Assignment statement

Code (left):

Statements and expressions
Red arrow points to next line.
Gray arrow points to the line just executed

Frames (right):

A name is bound to a value
In a frame, there is at most one binding per name

```
1 from operator import mul
2 def square(x):
3     return mul(x, x)
4 square(-2)
```

Built-in function

User-defined function

Global frame

Intrinsic name of function called

Local frame

Formal parameter bound to argument

Return value

Return value is not a binding!

```
1 from operator import mul
2 def square(x):
3     return mul(x, x)
4 square(square(3))
```

A name evaluates to the value bound to that name in the earliest frame of the current environment in which that name is found.

Evaluation rule for call expressions:

1. Evaluate the operator and operand subexpressions.
2. Apply the function that is the value of the operator subexpression to the arguments that are the values of the operand subexpressions.

Applying user-defined functions:

1. Create a new local frame with the same parent as the function that was applied.
2. Bind the arguments to the function's formal parameter names in that frame.
3. Execute the body of the function in the environment beginning at that frame.

Execution rule for def statements:

1. Create a new function value with the specified name, formal parameters, and function body.
2. Its parent is the first frame of the current environment.
3. Bind the name of the function to the function value in the first frame of the current environment.

Execution rule for assignment statements:

1. Evaluate the expression(s) on the right of the equal sign.
2. Simultaneously bind the names on the left to those values, in the first frame of the current environment.

Execution rule for conditional statements:

- Each clause is considered in order.
1. Evaluate the header's expression.
 2. If it is a true value, execute the suite, then skip the remaining clauses in the statement.

Evaluation rule for or expressions:

1. Evaluate the subexpression <left>.
2. If the result is a true value *v*, then the expression evaluates to *v*.
3. Otherwise, the expression evaluates to the value of the subexpression <right>.

Evaluation rule for and expressions:

1. Evaluate the subexpression <left>.
2. If the result is a false value *v*, then the expression evaluates to *v*.
3. Otherwise, the expression evaluates to the value of the subexpression <right>.

Evaluation rule for not expressions:

1. Evaluate <exp>; The value is True if the result is a false value, and False otherwise.

Execution rule for while statements:

1. Evaluate the header's expression.
2. If it is a true value, execute the (whole) suite, then return to step 1.

Dictionary Methods

```
>>> food = {"ham":10, "cheese":12}
>>> food["cheese"]
12
>>> "peanuts" in food
False
>>> food["peanuts"] = 7 # adds
key-value pair to food dict
>>> "peanuts" in food
True
>>> food["ham"] = food["ham"] + 1
>>> food["ham"]
11
>>> [(key, food[key]) for key in
food]
[('ham', 11), ('cheese', 12),
('peanuts', 7)]
```

List comprehensions

[<map exp> for <name> in <iter exp> if <filter exp>]

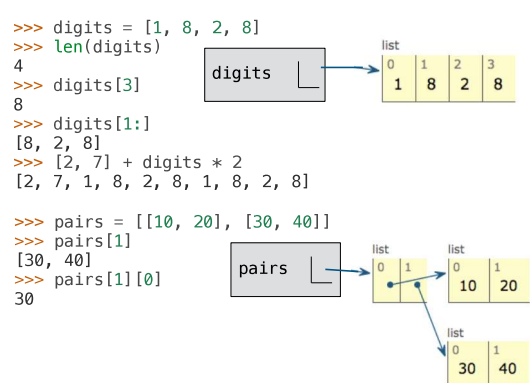
Short version: [<map exp> for <name> in <iter exp>]

A combined expression that evaluates to a list using this evaluation procedure:

1. Add a new frame with the current frame as its parent
2. Create an empty *result* list that is the value of the expression
3. For each element in the iterable value of <iter exp>:
 - A. Bind <name> to that element in the new frame from step 1
 - B. If <filter exp> evaluates to a true value, then add the value of <map exp> to the result list

List Methods

```
>>> lst = [8, 61]
>>> lst.append(10)
>>> lst
[8, 61, 10]
>>> lst.extend([2, 3])
>>> lst
[8, 61, 10, 2, 3]
>>> lst.insert(0, 88)
>>> lst
[88, 8, 61, 10, 2, 3]
>>> lst[1:3]
[8, 61]
>>> lst.pop(0)
88
>>> lst
[8, 61, 10, 2, 3]
>>> lst.remove(61)
>>> lst
[8, 10, 2, 3]
>>> lst.pop()
3
>>> lst
[8, 10, 2]
```

List Environment Diagram**Lists "Aggregate" Methods**

```
>>> lst = [-2, 4, 6]
>>> len(lst)
3
>>> sum(lst)
8
>>> min(lst)
-2
>>> max(lst, key=lambda x: -x)
-2
>>> lst = [(1, 9), (2, 5), (3, 4)]
>>> max(lst, key=lambda y: y[0] * y[1])
(3, 4)
```

Executing a for statement:

- for <name> in <expression>:
 <suite>
1. Evaluate the header <expression>, which must yield an iterable value (a list, tuple, iterator, etc.)
 2. For each element in that sequence, in order:
 - A. Bind <name> to that element in the current frame
 - B. Execute the <suite>

..., -3, -2, -1, 0, 1, 2, 3, 4, ...

range(-2, 2)

Length: ending value - starting value

Element selection: starting value + index

```
>>> list(range(-2, 2))
[-2, -1, 0, 1]
```

```
>>> list(range(4))
[0, 1, 2, 3]
```

List constructor

Range with a 0 starting value

Miscellaneous Operations

```
>>> 5 // 3
1
>>> 5 % 3
2
>>> 2 * 3
6
>>> 2 + 3
5
>>> 6 / 3
2.0
>>> min(2, 1, 4, 3)
1
>>> max(2, 1, 4, 3)
4
>>> abs(-2)
2
>>> pow(2, 3)
8
>>> len('word')
4
>>> print(1, 2)
1 2
```

Functional List Operations

goal: transform a list, and return a new result

map(function, list_of_inputs)
transform each item by a function.

function input: 1 argument (each item)

function output: "anything", a new item

map output: list of the same length, but possibly new values

filter(function, list_of_inputs)
keeps each item where the function is true.

function input: 1 argument (each item)

function output: boolean

filter output: list with possibly fewer items, but values are the same

reduce(function, list_of_inputs)
successively combine items.

function input: 2 arguments (current item, and the previous result)

function output: type should match the type of each item

reduce output: usually a "single" item

```
square = lambda x,y: x * y
```

A function

with formal parameters x and y
that returns the value of $"x * y"$

Evaluates to a function.
No "return" keyword!

Must be a single expression

```
def make_adder(n):
```

A function that returns a function

```
    """Return a function that takes one argument k and returns k + n."""
```

```
>>> add_three = make_adder(3)
```

```
>>> add_three(4)
```

7

The name `add_three` is
bound to a function

```
def adder(k):
```

```
    return k + n
```

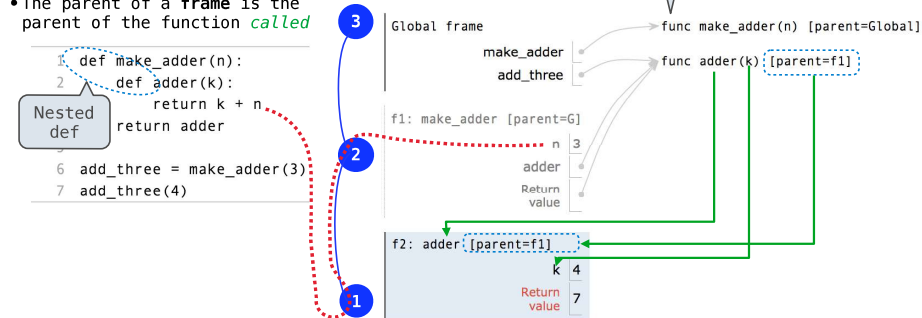
```
    return adder
```

A local
def statement

Can refer to names in
the enclosing function

- Every user-defined function has a **parent frame** (often global)
- The parent of a function is the frame in which it was **defined**
- Every local frame has a **parent frame** (often global)
- The parent of a frame is the parent of the function **called**

A function's signature
has all the information
to create a local frame



Python object system:

Idea: All bank accounts have a **balance** and an account **holder**;
the **Account** class should add those attributes to each of its instances

```
>>> a = Account('Jim')
>>> a.holder
'Jim'
>>> a.balance
0
```

A new instance
is created by
calling a class

An account instance

```
balance: 0
holder: 'Jim'
```

When a class is called:

1. A new instance of that class is created:
2. The `__init__` method of the class is called with the new object as its first argument (named `self`), along with any additional arguments provided in the call expression.

```
class Account:
    def __init__(self, account_holder):
        self.balance = 0
        self.holder = account_holder
    def deposit(self, amount):
        self.balance = self.balance + amount
        return self.balance
    def withdraw(self, amount):
        if amount > self.balance:
            return 'Insufficient funds'
        self.balance = self.balance - amount
        return self.balance
```

`__init__` is called
a constructor

`self` should always
be bound to an
instance of the
Account class or a
subclass of Account

Function call:
all arguments
within
parentheses

```
>>> type(Account.deposit)
<class 'function'>
>>> type(a.deposit)
<class 'method'>
```

```
>>> Account.deposit(a, 5)
```

10

```
>>> a.deposit(2)
```

12

Dot expression

Call expression

Method
invocation: One
object before the
dot and other
arguments within
parentheses

```
square = lambda x: x * x
```

VS

```
def square(x):
    return x * x
```

- Both create a function with the same domain, range, and behavior.
- Both functions have as their parent the environment in which they were defined.
- Both bind that function to the name `square`.
- Only the `def` statement gives the function an intrinsic name.

When a function is defined:

1. Create a **function value**: `func <name>(<formal parameters>)`
2. Its parent is the current frame.

```
f1: make_adder      func adder(k) [parent=f1]
```

3. Bind `<name>` to the **function value** in the current frame (which is the first frame of the current environment).

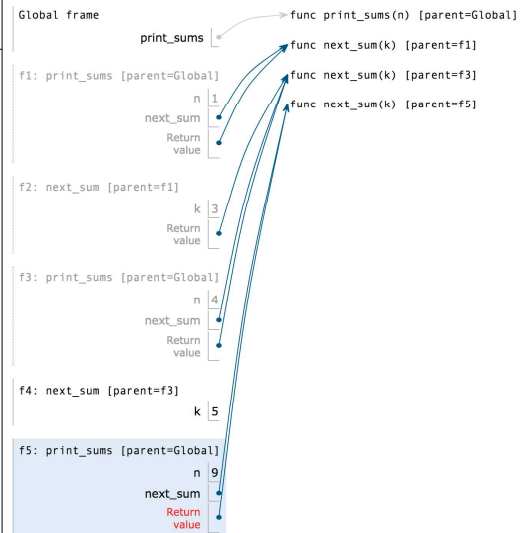
When a function is called:

1. Add a **local frame**, titled with the `<name>` of the function being called.
2. Copy the parent of the function to the **local frame**: [`parent=<label>`]
3. Bind the `<formal parameters>` to the arguments in the **local frame**.
4. Execute the body of the function in the environment that starts with the **local frame**.

```
1 def print_sums(n):
2     print(n)
3     def next_sum(k):
4         return print_sums(n+k)
5     return next_sum
6
7 print_sums(1)(3)(5)
```

Printed output:

```
1
4
9
```



False values so far: `0`, `False`, `''`, `None`
Anything value that's not false is true.

```
>>> if 0:
...     print('!')
>>> if 1:
...     print('!')
>>> if 1 and 0:
...     print('!')
>>> if 1 or 0:
...     print('!')
>>> if abs:
...     print('!')
>>> if 1 or 1/0:
...     print('!')
```

```
from operator import floordiv, mod
def divide_exact(n, d):
    """Return the quotient and remainder of dividing N by D.
```

```
>>> q, r = divide_exact(2012, 10)
>>> q
201
>>> r
2
```

```
"""Multiple assignment to two names
```

```
return floordiv(n, d), mod(n, d)"""
```

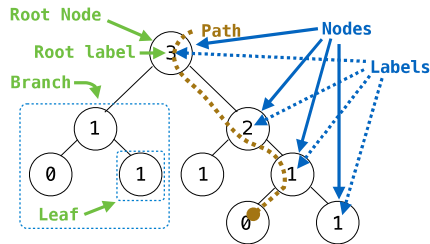
```
Two return values, separated by commas
```

Recursive description:

- A **tree** has a **root label** and a list of **branches**
- Each branch is a **tree**
- A tree with zero branches is called a **leaf**

Relative description:

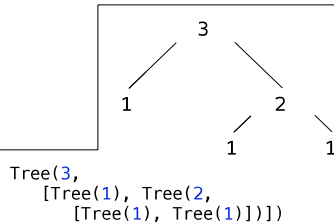
- Each location is a **node**
- Each **node** has a **label**
- One node can be the **parent/child** of another

**class Tree:**

```
def __init__(self, label, branches=[]):
    self.label = label
    for branch in branches:
        assert isinstance(branch, Tree)
    self.branches = list(branches)
```

Built-in `isinstance` function: returns True if **branch** has a class that is or inherits from **Tree**

```
def is_leaf(self):
    return not self.branches
```



```
def fib_tree(n):
    if n == 0 or n == 1:
        return Tree(n)
    else:
        left = fib_tree(n-2)
        right = fib_tree(n-1)
        fib_n = left.label + right.label
        return Tree(fib_n, [left, right])
```

Exceptions are raised with a `raise` statement.

```
raise <expr>
```

<expr> must evaluate to a subclass of `BaseException` or an

```
try:
    <try suite>
except <exception class> as <name>:
    <except suite>
```

```
>>> try:
    x = 1/0
except ZeroDivisionError as e:
    print('handling a', type(e))
    x = 0
handling a <class 'ZeroDivisionError'>
>>> x
0
```

The <try suite> is executed first.

If, during the course of executing the <try suite>, an exception is raised that is not handled otherwise, and

If the class of the exception inherits from <exception class>, then

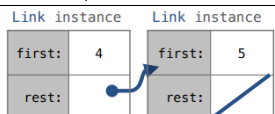
The <except suite> is executed, with <name> bound to the exception.

class Link:

```
empty = ()
def __init__(self, first, rest=empty):
    assert rest is Link.empty or isinstance(rest, Link)
    self.first = first
    self.rest = rest
```

```
def __repr__(self):
    if self.rest:
        rest = ', ' + repr(self.rest)
    else:
        rest = ''
    return 'Link(' + repr(self.first) + rest + ')'
```

```
def __str__(self):
    string = '('
    while self.rest is not Link.empty:
        string += str(self.first) + ', '
        self = self.rest
    return string + str(self.first) + ')'
```



```
>>> s = Link(4, Link(5))
>>> s
Link(4, Link(5))
>>> s.first
4
>>> s.rest
Link(5)
>>> print(s)
(4 5)
>>> print(s.rest)
(5)
>>> s.rest.rest is Link.empty
True
```

Anatomy of a recursive function:

- The **def statement header** is like any function
- Conditional statements check for **base cases**
- Base cases are evaluated **without recursive calls**
- Recursive cases are evaluated **with recursive calls**

```
def sum_digits(n):
    """Sum the digits of positive integer n."""
    if n < 10:
        return n
    else:
        all_but_last, last = n // 10, n % 10
        return sum_digits(all_but_last) + last
```

Recursive decomposition: finding simpler instances of a problem.

- E.g., `count_partitions(6, 4)`

Explore two possibilities:

- Use at least one 4
- Don't use any 4

Solve two simpler problems:

- `count_partitions(2, 4)`
- `count_partitions(6, 3)`

Tree recursion often involves exploring different choices.

```
def count_partitions(n, m):
    if n == 0:
        return 1
    elif n < 0:
        return 0
    elif m == 0:
        return 0
    else:
        with_m = count_partitions(n-m, m)
        without_m = count_partitions(n, m-1)
        return with_m + without_m
```

Python object system:

Idea: All bank accounts have a **balance** and an account **holder**; the **Account** class should add those attributes to each of its instances

A new instance is created by calling a class

```
>>> a = Account('Jim')
>>> a.holder
'Jim'
>>> a.balance
0
```

An account instance

```
balance: 0 holder: 'Jim'
```

When a class is called:

1. A new instance of that class is created:

2. The `__init__` method of the class is called with the new object as its first argument (named `self`), along with any additional arguments provided in the call expression.

class Account:

```
def __init__(self, account_holder):
    self.balance = 0
    self.holder = account_holder
def deposit(self, amount):
    self.balance = self.balance + amount
    return self.balance
def withdraw(self, amount):
    if amount > self.balance:
        return 'Insufficient funds'
    self.balance = self.balance - amount
    return self.balance
```

`__init__` is called a constructor

`self` should always be bound to an instance of the Account class or a subclass of Account

Function call: all arguments within parentheses

```
>>> type(Account.deposit)
<class 'function'>
>>> type(a.deposit)
<class 'method'>
```

Method invocation: One object before the dot and other arguments within parentheses

```
>>> Account.deposit(a, 5)
10
>>> a.deposit(2)
12
```

Call expression

Dot expression

<expression> . <name>

The <expression> can be any valid Python expression.

The <name> must be a simple name.

Evaluates to the value of the attribute looked up by <name> in the object that is the value of the <expression>.

To evaluate a dot expression:

1. Evaluate the <expression> to the left of the dot, which yields the object of the dot expression
2. <name> is matched against the instance attributes of that object; if an attribute with that name exists, its value is returned
3. If not, <name> is looked up in the class, which yields a class attribute value
4. That value is returned unless it is a function, in which case a bound method is returned instead

Assignment statements with a dot expression on their left-hand side affect attributes for the object of that dot expression

- If the object is an instance, then assignment sets an instance attribute
- If the object is a class, then assignment sets a class attribute

Account class attributes

```
interest: 0.02 0.04 0.05
(withdraw, deposit, __init__)
```

Instance attributes of jim_account

```
balance: 0
holder: 'Jim'
interest: 0.08
```

Instance attributes of tom_account

```
balance: 0
holder: 'Tom'
```

```
>>> jim_account = Account('Jim')
>>> tom_account = Account('Tom')
>>> tom_account.interest
0.02
>>> jim_account.interest
0.02
>>> Account.interest = 0.04
>>> tom_account.interest
0.04
>>> jim_account.interest
0.04
```

```
>>> jim_account.interest = 0.08
>>> jim_account.interest
0.08
>>> tom_account.interest
0.04
>>> Account.interest = 0.05
>>> tom_account.interest
0.05
>>> jim_account.interest
0.08
```

class CheckingAccount(Account):

```
"""A bank account that charges for withdrawals."""
withdraw_fee = 1
interest = 0.01
def withdraw(self, amount):
```

```
    return Account.withdraw(self, amount + self.withdraw_fee)
    or
    return super().withdraw(
        amount + self.withdraw_fee)
```

To look up a name in a class:

1. If it names an attribute in the class, return the attribute value.
2. Otherwise, look up the name in the base class, if there is one.

```
>>> ch = CheckingAccount('Tom') # Calls Account.__init__
>>> ch.interest # Found in CheckingAccount
0.01
>>> ch.deposit(20) # Found in Account
20
>>> ch.withdraw(5) # Found in CheckingAccount
14
```



```
iter(iterable):
    Return an iterator
    over the elements of
    an iterable value
next(iterator):
    Return the next element

>>> s = [3, 4, 5]
>>> t = iter(s)
>>> next(t)
3
>>> next(t)
4

>>> d = {'one': 1, 'two': 2, 'three': 3}
>>> k = iter(d)
>>> next(k)
'one'
>>> next(k)
'two'
```

A *generator function* is a function that *yields* values instead of *returning* them.

```
>>> def plus_minus(x):
...     yield x
...     yield -x

>>> t = plus_minus(3)
>>> next(t)
3
>>> next(t)
-3

def a_then_b(a, b):
    yield from a
    yield from b

>>> list(a_then_b([3, 4], [5, 6]))
[3, 4, 5, 6]
```

Efficiency

- Constant growth:** $O(1)$
E.g., accessing a value from a dictionary
Increasing n doesn't affect time
- Logarithmic growth:** $O(\log(n))$
E.g., binary search
Doubling n only increments time by a constant
- Linear growth:** $O(n)$
E.g., iterating over a list of length n
Incrementing n increases time by a constant
- Quadratic growth:** $O(n^2)$
E.g., finding all pairs of a list of integers (double for loop)
Incrementing n increases time by n times a constant
- Exponential growth:** $O(b^n)$
E.g., recursive fib
Incrementing n multiplies time by a constant

```
def perms(lst):
    """
    Generates the permutations of lst one by one.
    """
    >>> perms = perms([1, 2, 3])
    >>> p = list(perms)
    >>> p.sort()
    >>> p
    [[1, 2, 3], [1, 3, 2], [2, 1, 3], [2, 3, 1], [3, 1, 2], [3, 2, 1]]
    """
    if lst == []:
        yield []
    else:
        for perm in perms(lst[1:]):
            for i in range(len(lst)):
                yield perm[:i] + [lst[0]] + perm[i:]
```

A table has columns and rows

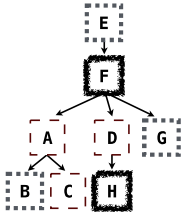
Latitude	Longitude	Name
38	122	Berkeley
42	71	Cambridge
45	93	Minneapolis

A column has a name and a type

A row has a value for each column

```
SELECT [expression] AS [name], [expression] AS [name], ... ;
SELECT [columns] FROM [table] WHERE [condition] ORDER BY [order];
```

```
CREATE TABLE parents AS
SELECT "abraham" AS parent, "barack" AS child
UNION
SELECT "abraham"      , "clinton"
UNION
SELECT "delano"       , "herbert"
UNION
CREATE TABLE dogs AS
SELECT "abraham" AS name, "long" AS fur UNION
SELECT "barack"    , "short"  UNION
SELECT "clinton"   , "long"   UNION
SELECT "delano"    , "long"   UNION
SELECT "eisenhower", "short" UNION
SELECT "fillmore"  , "curly"  UNION
SELECT "grover"     , "short"  UNION
SELECT "herbert"   , "curly";
```



```
SELECT a.child AS first, b.child AS second
FROM parents AS a, parents AS b
WHERE a.parent = b.parent AND a.child <
```

First	Second
barack	clinton
abraham	delano
abraham	grover
delano	grover

The number of groups is the number of unique values of an expression

A *having* clause filters the set of groups that are aggregated

```
select weight/legs, count(*) from animals
group by weight/legs
having count(*)>1;
```

weight/legs	count(*)
5	2
2	2

weight/legs=5
weight/legs=2
weight/legs=2
weight/legs=3
weight/legs=5
weight/legs=6000

kind	legs	weight
dog	4	20
cat	4	10
ferret	4	10
parrot	2	6
penguin	2	10
t-rex	2	12000